

Pump Integration API Documentation

Welcome to the **Pump Integration API Reference Guide**. This documentation is designed for device developers building apps and firmware for fuel pump hardware.

This guide details how a pump device authenticates with the backend system, initiates sales transactions, and uploads final fuel dispense logs.

1. Overview & Base Environments

All API requests and responses communicate using standard **JSON** payloads.

- **Production Base URL:** `https://<your-host>`
- **Local Development URL:** `http://localhost:8080`

Response Envelope

Every response from the server follows a standardized response envelope:

```
JSON
{
  "status": true
  "message": "Human readable message"
  "data":
}
```

- **Success Status:** When an operation succeeds, status is true and the requested payload is housed inside data.
- **Error Status:** On failure, status is false, data is typically absent or null, and the message string provides the error reason.

Global HTTP Status Codes

HTTP Status Code	Description	Corresponding Exception
400 Bad Request	Validation or business logic rule validation failed.	CustomException
401 Unauthorized	Authentication, device token verification, or signature verification failed.	UnauthorizedException

404 Not Found	The requested transaction, pump, or resource could not be found.	NotFoundException
---------------	--	-------------------

2. Device Lifecycle & Workflow

The integration follows a structured sequence across four main stages:



```
API Server |
| (Dispense Log) | <-----
(DISPENSED) |
|----- Transaction successfully closed
|-----
```

The pump hardware device only executes three API actions: **Activate**, **Authorise**, and **Upload**. Setting up, managing, or revoking pump devices is handled externally by the station administration dashboard.

3. Authentication & Request Signing

The activation endpoint (/api/pump/activate) is completely public. However, **all subsequent calls** (authorise and upload) require explicit request signing using cryptographic HMAC validation.

Required Custom HTTP Headers

```
HTTP
X-API-Key: bal_live_XXXXXXXXXXXXXXXXXXXXX
X-Device-Id: device_001
X-Timestamp: 2026-06-13T19:08:31Z
X-Signature: <hex-encoded-HMAC-SHA256-signature>
Content-Type: application/json
```

Signature Generation Rules

1. **Timestamp Freshness:** The X-Timestamp header value must be formatted as an ISO-8601 UTC string (yyyy-MM-dd'T'HH:mm:ss'Z'). The server clock and device clock must align within a strict **5-minute window**, or the system rejects the packet.
2. **Payload Matching:** Compute your signature using the **exact raw byte stream** of the outbound JSON request body. Do not alter spacing, prettify, or re-serialize data after generating your hash, as minor format differences break signature validity.
3. **Formula:**
$$\text{X-Signature} = \text{HMAC_SHA256}(\text{signingSecret}, \text{X-Timestamp} + \text{"."} + \text{rawRequestBody})$$

Reference Implementation (Node.js)

JavaScript

```
const crypto = require("crypto")
```

```
// 1. Generate formatted ISO Timestamp
```

```
const timestamp = new Date().toISOString().replace(/\.d{3}Z$/, "Z")
```

```
// 2. Compute HMAC SHA256 signature over raw stringified body
```

```
const signature = crypto
```

```
    .createHmac("sha256", signingSecret)
```

```
    .update(timestamp + "." + rawRequestBody)
```

```
    .digest("hex")
```

Authentication Failures

HTTP	Error Message	Underlying Root Cause
401	Missing pump authentication headers	X-Api-Key or X-Device-Id headers are missing.
401	Missing request signature headers	X-Timestamp or X-Signature headers are missing.
401	Invalid API key	The given API key was not recognized by the platform.
401	Credential is not a pump device	The secret key belongs to an actor profile type that is not a pump.
401	Pump credentials have been revoked	Station admins marked this specific device's access as revoked.
401	Pump is not active	Attempting to sign requests

		prior to running activation.
401	Device id does not match credential	The provided X-Device-Id does not align with the validated API key.
401	Invalid request timestamp	The X-Timestamp header is malformed or non-ISO-8601 standard.
401	Request timestamp is not fresh	The device clock skews by more than 5 minutes from server UTC time.
401	Invalid request signature	The local signature computation does not match the server-calculated value.

4. Endpoint Reference

4.1 Device Activation

POST /api/pump/activate

Used once during onboarding. Redeems a single-use activation token code issued by the service portal in exchange for long-lived access parameters.

- **Authentication:** None (Public Endpoint)

Request Payload

JSON

```
{
  "activationCode" "PMP-O8l6zj"
  "deviceId" "device_001"
}
```

- `activationCode` (*String, Required*): The physical registration code mapped out by the fuel station.
- `deviceId` (*String, Required*): A unique hardware identifier defined by your hardware client software. This value must accompany all future signed message configurations.

Success Response (200 OK)

```

JSON
{
  "status": true
  "message": "Pump activated successfully"
  "data": {
    "deviceId": "device_001"
    "pumpId": "7f108b57-7559-4837-8dfb-33c7aac7d632"
    "apiKey": "bal_live_XXXXXXXXXXXXXXXXXXXX"
    "signingSecret": "sec_XXXXXXXXXXXXXXXXXXXX"
  }
}

```

⚠ CRITICAL SECURITY NOTE: Cache `apiKey` and `signingSecret` inside secure hardware-encrypted storage immediately. These cryptographic parameters are emitted **exactly once** on activation. If accidentally wiped, the station must revoke and reissue a new activation code.

Endpoint Error Codes

HTTP	Error Message	Underlying Root Cause
400	Activation code is required	The request payload omitted the <code>activationCode</code> attribute.
400	Device id is required	The request payload omitted the <code>deviceId</code> attribute.

400	Invalid activation code	The activation identifier structure could not be discovered in the system.
400	Activation code has already been used	This token code was successfully redeemed previously.
400	Activation code has expired	Token lifespan exceeded its allowed time-to-live parameter.
400	Pump credentials have been revoked	The base infrastructure profile was decommissioned by admin operations.

4.2 Start a Sale / Transaction Authorization

POST /api/pump/authorise

Initiates an order request to initialize digital payments. The response generates a standard Paystack target checkout hyperlink to render into a payment QR code on the physical pump monitor.

- **Authentication:** Signed Headers Required (Section 3)

Request Payload

JSON

```
[
  {
    "pumpId" "7f108b57-7559-4837-8dfb-33c7aac7d632"
    "transactionId" "txn_0007"
    "amount" 7000
    "expectedLitres" 10
    "fuelType" "PETROL"
  }
]
```

- `transactionId` (*String, Required*): Locally generated identifier. Acts as an **idempotency key**. If requests fail mid-transit, re-submitting an identical ID safely yields the existing checkout object instead of producing duplicate data duplicates.
- `fuelType` (*String, Required*): Must be one of the following valid codes: PETROL, KEROSENE, DIESEL, COOKING_GAS.
- `amount` (*Numeric, Required*): The total target monetary value. Must exactly match the product calculation formula: $\text{\$}\{\text{expectedLitres}\} \times \{\text{stationPricePerUnit}\}\text{\$}$.

Success Response (200 OK)

```

JSON
{
  "status": true,
  "message": "Transaction authorised",
  "data": {
    "status": "PENDING_PAYMENT",
    "transactionId": "txn_0007",
    "paymentReference": "BPM-5397913a552441f4aa853eb60ca47d05",
    "authorizationUrl": "https://checkout.paystack.com/pkjo1mam84zn2sx",
    "expiresAt": "2026-06-13T19:28:42Z"
  }
}

```

- Convert the `authorizationUrl` string attribute directly into a dynamic QR graphics asset on-screen for customer interaction.
- Retain `transactionId` and `paymentReference` in active device tracking memory; both parameters are mandatory inputs for the subsequent log collection cycle.
- *Note:* Retrying an identical transaction payload returns: `"message": "Transaction already initiated"`.

Endpoint Error Codes

HTTP	Error Message	Underlying Root Cause
400	pumpId is required / transactionId is required ...	One or more structural input properties are missing.

400	amount must be greater than zero	Entered value evaluates to a negative number or zero.
400	expectedLitres must be greater than zero	Entered volume metric evaluations are invalid.
400	Invalid request body	Payload stream text failed standard JSON schema validation parsing.
401	pumpId does not match authenticated device	Validated device profile credentials map to a different pump index identifier.
400	Fuel type not available at station: PETROL	The station profile configuration lacks pricing rules for this inventory type.
400	PETROL is currently out of stock	Inventory flags for this specific fuel code are set to zero.
400	Fuel station has an invalid price per unit for PETROL	The unit cost evaluating rule structure dropped below zero or is unassigned.
400	Amount mismatch for PETROL...	The request's total amount does not match $\text{\$} \times \text{\$}$.
400	Unable to initialize pump payment	Handshake interactions or gateway negotiation tasks with Paystack failed.

4.3 Upload Dispense Log

POST /api/pump/transactions/upload

Transmits real-time physical meter data readings upon hardware operation completion. This

step updates the transaction status from paid to **fulfilled**.

- **Authentication:** Signed Headers Required (Section 3)

Request Payload

JSON

```
{
  "pumpId" "7f108b57-7559-4837-8dfb-33c7aac7d632"
  "transactionId" "txn_0007"
  "paymentReference" "BPM-5397913a552441f4aa853eb60ca47d05"
  "actualLitresDispensed" 10
  "startedAt" "2026-06-13T19:10:00Z"
  "completedAt" "2026-06-13T19:12:00Z"
}
```

- `actualLitresDispensed` (*Numeric, Required*): Exact fractional total quantity registered by physical fluid line measurement systems.

Success Response (200 OK)

JSON

```
{
  "status" true
  "message" "Transaction recorded"
  "data"
    "status" "DISPENSED"
    "transactionId" "txn_0007"
    "paymentReference" "BPM-5397913a552441f4aa853eb60ca47d05"
}
```

- **Idempotency Handling:** If transmission dropouts happen during upload processing, re-sending identical payloads safe-guards records, yielding success outputs without

double-processing transaction metrics.

Endpoint Error Codes

HTTP	Error Message	Underlying Root Cause
400	pumpId is required / transactionId is required ...	One or more structural input properties are missing.
400	actualLitresDispensed cannot be negative	Provided metric value contains sub-zero characters.
401	pumpId does not match authenticated device	Validated device profile credentials map to a different pump index identifier.
404	Pump transaction not found	The target transactionId record index was not found on the server.
401	Pump does not own this transaction	The transaction index maps to an alternative active pump node.
400	Payment reference does not match this transaction	The paymentReference string value contradicts original order maps.
400	Payment has not been confirmed for this transaction	The customer's payment process remains unverified. Do not dispense fuel.

5. Summary Reference Cheat Sheet

Endpoint Quick Map

API Endpoint Route	Method Type	Authentication Security Strategy
/api/pump/activate	POST	None (Public Token Validation)
/api/pump/authorise	POST	Active Secure API Key Verification + HMAC Signature Verification
/api/pump/transactions/upload	POST	Active Secure API Key Verification + HMAC Signature Verification

Status State Machine Transitions (data.status)

1. PENDING_PAYMENT

The checkout context has been initiated. The application should render the checkout QR code and await confirmation.

2. DISPENSED

Payment confirmation was successfully processed by webhooks, and physical meter volume values have been logged. The transaction is closed.